Práctica 4.1
Positividad y competencias.

Docente: Gustavo Landfried

Inferencia Bayesiana Causal 1
2do cuatrimestre 2026
UNSAM

Índice

1. Positividad en la simulación de Hill **2**
1.1. El modelo . . . 2
1.2. El zoológico en cada k . . . 3
1.3. Resultados . . . 3

2. La competencia ACIC 2016 **5**
2.1. Los datos . . . 5
2.2. Solapamiento con 58 covariables . . . 5
2.3. El zoológico en los dos escenarios . . . 6
2.4. Estimado vs verdadero . . . 7

3. Entrega **8**

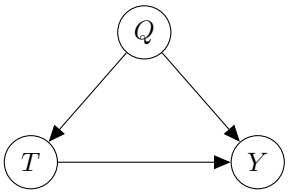
1. Positividad en la simulación de Hill

En la teórica vimos que, bajo ignorabilidad condicional, el efecto causal se obtiene con la adjustment formula, y que el zoológico de algoritmos son distintas formas de estimar sus dos caras: la regresión y la ponderación. Pero la adjustment formula supone además positividad: que todo individuo tenga alguna chance de recibir cada tratamiento, $0 < P(T = 1|q) < 1$. Sin positividad fallan las dos caras. La ponderación divide por $P(t|q)$, que vale cero, y la regresión necesita $\mathbb{E}[Y|t, q]$ en valores de q donde nunca se asignó el tratamiento t.

En el caso de la fintech, el préstamo se asignaba con una función determinista de variables conocidas. El conjunto de control era correcto y aun así el efecto no se podía obtener de los datos. En la simulación de Hill de la teórica, en cambio, la positividad se cumplía en los papeles pero no en la muestra: en los bordes había una zona sin solapamiento, donde solo había controles o solo tratados. En esta práctica vamos a controlar el grado de positividad para ver qué le pasa a cada algoritmo del zoológico a medida que se pierde.

1.1. El modelo

Usamos la misma simulación de la teórica, en la que Q es una causa común del tratamiento T y del resultado Y.



La única diferencia es que la asignación del tratamiento depende de un parámetro k que controla la positividad.

$$\begin{aligned} P(q) &= \tfrac{1}{2}\,\mathcal{N}(q|20, 10^2) + \tfrac{1}{2}\,\mathcal{N}(q|40, 10^2) \\ P(t|q, k) &= \text{Bernoulli}\big(t \,|\, \sigma(k\,(q - 30))\big) \\ P(y|q, t) &= \mathcal{N}\big(y \,|\, \mu_t(q), 1\big) \end{aligned} \tag{1}$$

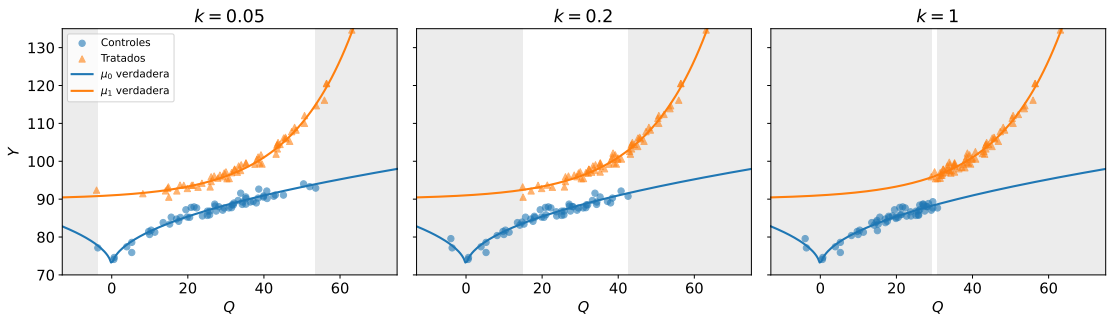
donde $\sigma(x) = 1/(1+e^{-x})$ y las curvas de cada grupo son $\mu_0(q) = 72+3\sqrt{|q|}$ y $\mu_1(q) = 90+e^{0.06\,q}$. La simulación de la teórica es el caso $k = 0.2$. Con k chico la probabilidad de tratamiento es casi la misma para todos, como en un experimento aleatorizado, y el solapamiento es casi total. Con k grande la asignación es casi determinista, como en el caso de la fintech: los individuos con q menor a 30 casi nunca son tratados y los de q mayor a 30 casi siempre.

El efecto de cada individuo es $\tau(q) = \mu_1(q) - \mu_0(q)$, que cambia con q. Como en la teórica, el ATE verdadero es el promedio de $\tau(q_i)$ sobre los individuos de la muestra.

Objetivo: simular con distintos grados de positividad. En el archivo `practica_4_1.py` completá las funciones `asignacion` y `simular`.

1. `asignacion(q, k)` devuelve $P(T = 1|q) = \sigma(k\,(q - 30))$ para un array `q`.
2. `simular(k, semilla, n=120)` devuelve los arrays `q`, `t` e `y` del modelo. Para todo lo aleatorio usá un solo generador, `rng = np.random.default_rng(semilla)`.
3. Corré `figura_positividad_datos`, que ya grafica los datos con $k \in \{0.05, 0.2, 1\}$ y sombrea la zona sin solapamiento.

El gráfico resultante se debería ver de la siguiente forma:



1.2. El zoológico en cada k

El archivo `algoritmos.py` tiene todos los algoritmos de la teórica. En la práctica corremos diez: diferencia ingenua, matching, IPW, S-Learner, T-Learner, doubly robust, DML, Causal Forest, BART y BCF. Las redes neuronales, TARNet y Dragonnet, también están en `algoritmos.py`, pero no las corremos porque duplicarían el tiempo de la corrida: quedan para quien quiera explorarlas. Todos tienen la misma interfaz. Reciben la matriz de covariables `X` (en Hill, una sola columna con q), el tratamiento `t` y el resultado `y`, y devuelven un diccionario con el ATE estimado (`"ate"`), su intervalo del 95 % (`"ate_inf"` y `"ate_sup"`) y el efecto estimado de cada individuo (`"tau"`), que vale `None` en los algoritmos que solo dan un ATE. La función `correr` de la plantilla los aplica a todos sobre los mismos datos. Como ejemplo de cómo se usa cada algoritmo, la carpeta `ejemplo_teorica/` trae `hill.py`, el script que generó las figuras de la simulación de Hill de la teórica. No es parte de la entrega: se corre aparte y sus figuras quedan en esa carpeta.

```
cd ejemplo_teorica
MPLBACKEND=Agg PYTHONPATH=.. python hill.py
```

Para comparar los algoritmos usamos las tres métricas de la teórica. El error en el ATE, $|\widehat{\text{ATE}} - \text{ATE}|$. El PEHE, que mide cuánto se equivoca el algoritmo en el efecto de cada individuo,

$$\text{PEHE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(\hat{\tau}_i - \tau_i\right)^2} \tag{2}$$

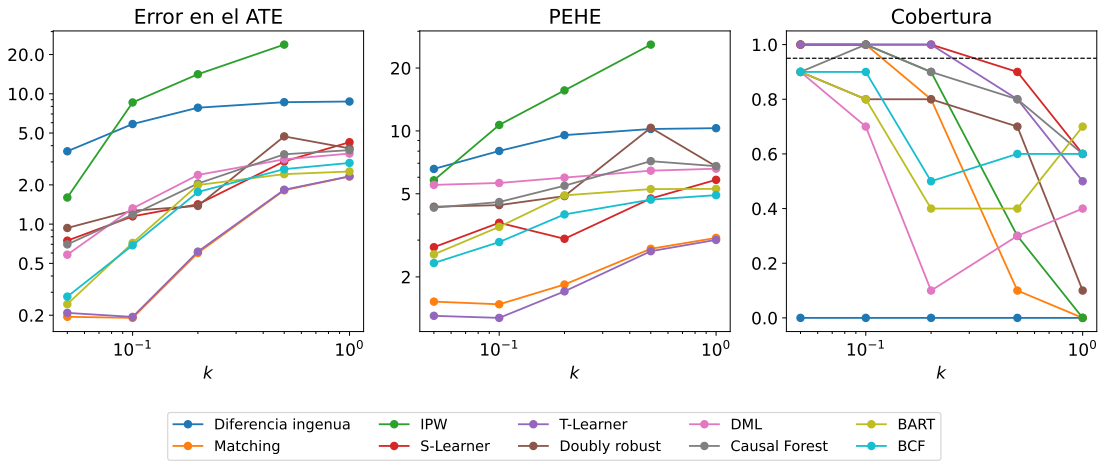
donde, en los algoritmos que solo dan un ATE, $\hat{\tau}_i$ es ese ATE repetido. Y la cobertura, la proporción de réplicas en las que el intervalo del 95 % contiene al ATE verdadero. Si los intervalos son honestos, la cobertura tiene que estar cerca de 0.95. Con 10 réplicas la cobertura solo toma valores 0, 0.1, ..., 1, y aun con intervalos honestos puede salir 0.8 o menos (pasa con probabilidad cercana a 0.09): las diferencias chicas no dicen mucho.

Objetivo: medir a cada algoritmo.

1. Completá `metricas(r, tau_verdadero)`, que recibe el resultado `r` de un algoritmo y el efecto verdadero `tau_verdadero` de cada individuo, y devuelve un diccionario con las claves `"error_ate"`, `"pehe"` y `"cubre"` (si el intervalo contiene al ATE verdadero).
2. La función `experimento_positividad` ya recorre $k \in \{0.05, 0.1, 0.2, 0.5, 1\}$ con 10 réplicas cada uno, corre todos los algoritmos y guarda las métricas en `positividad.csv`.

1.3. Resultados

Objetivo: interpretar. La figura muestra el promedio de cada métrica sobre las réplicas, en función de k. El gráfico resultante se debería ver de la siguiente forma:



1. ¿Por qué IPW es el primero en romperse a medida que crece k? ¿Qué pasa con sus estimaciones cuando $k = 1$?
2. ¿Qué hacen BART y BCF en la zona sin datos? ¿Qué pasa con la cobertura de sus intervalos?
3. En la corrida de referencia, con una sola covariable, matching y T-Learner son los que menos se equivocan en todos los k. ¿Por qué? ¿Esperarías lo mismo con 58 covariables?
4. Con efecto heterogéneo, el DML parcialmente lineal no estima el ATE sino

$$\frac{\sum_i e(q_i)(1-e(q_i))\,\tau(q_i)}{\sum_i e(q_i)(1-e(q_i))}$$

   ¿Cómo cambian esos pesos cuando crece k, y qué individuos dejan de contar?
5. ¿Algún algoritmo recupera bien el efecto con $k = 1$? ¿Por qué esto no contradice lo que vimos con el caso de la fintech?

2. La competencia ACIC 2016

En la teórica vimos que las competencias de inferencia causal ponen a prueba a los algoritmos sobre datos simulados con efecto verdadero conocido. En la competencia de la Atlantic Causal Inference Conference de 2016 se tomaron las covariables reales de un estudio (4802 individuos y 58 covariables) y se simularon el tratamiento y el resultado en 77 escenarios, con 100 réplicas cada uno (Dorie, Hill, Shalit, Scott y Cervone 2019, *Automated versus Do-It-Yourself Methods for Causal Inference: Lessons Learned from a Data Analysis Competition*, Statistical Science 34(1)). Los escenarios varían el modelo del tratamiento, la proporción de tratados, el solapamiento, el modelo del resultado, cuánto se parecen las covariables que importan para el tratamiento y para el resultado (alineación) y la heterogeneidad del efecto. Los equipos estimaron el efecto en los tratados; nosotros, como en la teórica, estimamos el ATE de la muestra.

2.1. Los datos

Vamos a usar dos escenarios que son iguales en todo salvo el solapamiento.

Escenario	Tratamiento	Tratados	Solapamiento	Resultado	Alineación	Heterogeneidad
4	polinomial	0.35	total	exponencial	0.75	alta
7	polinomial	0.35	limitado	exponencial	0.75	alta

La columna Tratados es el parámetro de la simulación. Limitar el solapamiento baja la proporción de tratados que se observa: en el escenario 4 es de alrededor de 0.35 y en el 7, de alrededor de 0.29.

Los datos se bajan corriendo `make` en la carpeta `datos/`. El archivo pesa 1.2 GB, aunque solo se extraen las covariables (`data_cf_all/x.csv`) y las carpetas de los dos escenarios (`data_cf_all/4/` y `data_cf_all/7/`). Las covariables categóricas están codificadas con letras (`A`, `B`, ...). Cada réplica es un archivo `zymu_*.csv` con el tratamiento (`z`), el resultado de cada individuo bajo control y bajo tratamiento (`y0` e `y1`) y sus valores esperados (`mu0` y `mu1`). Como tenemos los dos resultados potenciales, el efecto verdadero de cada individuo es `mu1 - mu0`. Al pasar las categóricas a indicadoras, las 58 covariables se convierten en 82 columnas: `X` queda de 4802 x 82.

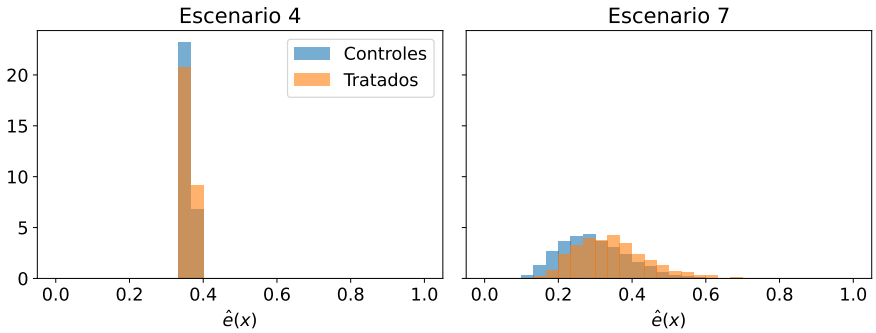
Objetivo: cargar una réplica.

1. Completá `cargar_acic(carpeta, escenario, replica)` siguiendo su docstring: covariables con las categóricas pasadas a indicadoras y las continuas estandarizadas, tratamiento, resultado observado y efecto verdadero.
2. Los nombres de los archivos tienen identificadores arbitrarios: la réplica es la posición del archivo cuando se los ordena por el número de su nombre.

2.2. Solapamiento con 58 covariables

Con una sola covariable podíamos ver la zona sin solapamiento en el plano Q, Y. Con 58 covariables ya no hay un plano para mirar. Pero el propensity score resume todas las covariables en un solo número, $\hat{e}(x) = \hat{P}(T = 1|x)$, y el solapamiento se ve comparando su distribución en tratados y en controles.

Objetivo: ver el solapamiento. La función `figura_acic_propension` estima el propensity con la regresión logística de `algoritmos.py` en la primera réplica de cada escenario. El gráfico resultante se debería ver de la siguiente forma:



El propensity score de una persona es la probabilidad de que reciba el tratamiento dadas sus covariables, $e(x) = P(T = 1|x)$. En los datos no lo vemos: lo estimamos con un modelo, en este caso una regresión logística, y el histograma muestra esas estimaciones $\hat{e}(x)$ separadas en tratados y controles. Si los dos histogramas se superponen, para cada tipo de persona hay gente comparable en el otro grupo: hay solapamiento.

En el escenario 4, los que armaron la competencia hicieron que el tratamiento dependiera de las covariables, pero de forma débil. Las covariables casi no permiten adivinar quién va a ser tratado, así que el modelo le asigna a todos prácticamente la misma probabilidad: la proporción de tratados, alrededor de 0.36. Por eso se ve una sola barra alta cerca de 0.375. Es como el caso de k chico en la simulación de Hill: casi un experimento aleatorizado. Pero solo casi: la diferencia ingenua igual da un efecto sesgado en este escenario, porque el resultado depende mucho de esas mismas covariables y hasta un desbalance chico entre los grupos mueve bastante los promedios.

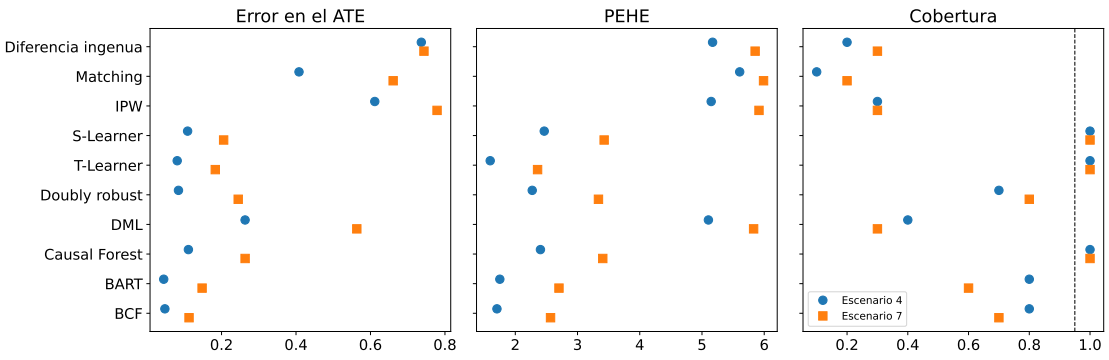
En el escenario 7 agregaron a la asignación del tratamiento un término fuerte, que hace que ciertos perfiles de personas casi nunca sean tratados. Incluso exigieron que más del 20 % de los controles tenga menos probabilidad de tratamiento que el tratado con menor probabilidad, es decir, que exista una región con solo controles. Ahora las covariables sí permiten adivinar bastante quién es tratado. Por eso el modelo estima probabilidades distintas para distintas personas, entre 0.1 y 0.7, y los controles quedan corridos hacia valores más bajos que los tratados. Es como subir k en Hill.

Si existe una región con solo controles, ¿por qué no aparecen personas con $\hat{e}(x)$ cerca de 0? Porque la regresión logística es un modelo lineal en la escala logit, y el término que crea esa región no lo es: el modelo no lo puede representar del todo y suaviza los extremos. El histograma del propensity estimado es tan bueno como el modelo que lo estimó. Una barra angosta no prueba que la asignación haya sido al azar, y la ausencia de valores cerca de 0 o de 1 no prueba que haya positividad.

1. ¿En qué escenario las covariables distinguen mejor a tratados de controles? ¿Qué significa eso para el solapamiento?
2. ¿Hay individuos con $\hat{e}(x)$ cercano a 0 o a 1? ¿Alcanza esta figura para afirmar que hay positividad, si el propensity se estimó con una regresión logística?

2.3. El zoológico en los dos escenarios

Objetivo: comparar. La función `experimento_acic` corre todos los algoritmos sobre 10 réplicas de cada escenario y guarda las métricas en `acic.csv`. De la primera réplica de cada escenario guarda además el efecto estimado de cada individuo, en `acic_tau.csv`, que se usa en la sección siguiente. El gráfico resultante se debería ver de la siguiente forma:

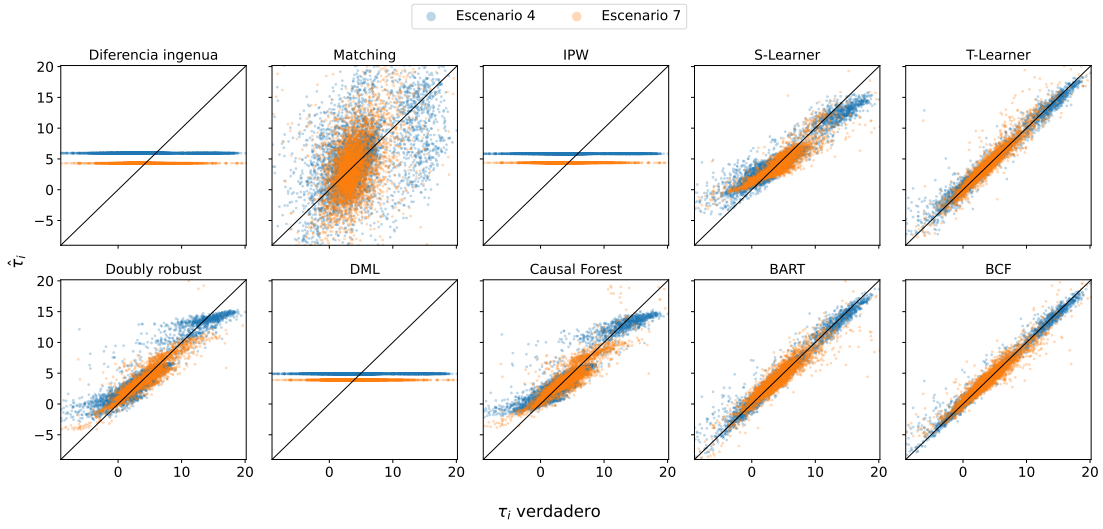


1. ¿Se repite lo que vimos en Hill al perder solapamiento? ¿Qué algoritmos son los más afectados?
2. Dorie y otros (2019) concluyeron que ganaban los métodos que modelan con flexibilidad la superficie de respuesta. ¿Se ve lo mismo acá?
3. ¿Qué intervalos cubren lo que prometen? Compará con la cobertura que vimos en la teórica sobre IHDP.

2.4. Estimado vs verdadero

Las métricas promedian sobre los individuos y sobre las réplicas. Para ver qué hace cada algoritmo con la heterogeneidad del efecto, comparamos el efecto estimado de cada individuo, $\hat{\tau}_i$, con su efecto verdadero, τ_i, como hicimos en la teórica con IHDP. Si un algoritmo acierta, sus puntos caen sobre la diagonal.

Objetivo: mirar el efecto de cada individuo. La función `figura_acic_tau` grafica $\hat{\tau}_i$ contra τ_i en la primera réplica de cada escenario, con un panel por algoritmo, a partir de `acic_tau.csv`. Unos pocos individuos del escenario 7, con efectos de hasta 200, quedan fuera de los ejes. El gráfico resultante se debería ver de la siguiente forma:



1. ¿Qué algoritmos aparecen como una recta horizontal y por qué? ¿Qué se pierde cuando solo se estima el ATE?

2. ¿Qué algoritmos siguen la diagonal y cuáles aplanan los efectos extremos? ¿Por qué el S-Learner tiende a aplanarlos?
3. ¿Por qué los $\hat{\tau}_i$ de matching forman una nube?
4. ¿Qué cambia en el escenario 7, con solapamiento limitado?

3. Entrega

En tu repositorio subí:

- `practica_4_1.py` con las funciones completas;
- `positividad.csv` y `acic.csv`, los resultados de la primera etapa;
- las cinco figuras en PNG (el script las guarda en PDF y en PNG);
- `practica_4_1.md` con las figuras PNG incrustadas y las respuestas a las preguntas.

No subas la carpeta `datos/` (pesa 69 MB y se vuelve a bajar con `make`), ni `acic_tau.csv` (pesa unos 4 MB), ni los PDF de las figuras, ni la carpeta `ejemplo_teorica/`.

El código usa numpy, pandas, scipy, matplotlib, scikit-learn, EconML, `stochtree` y PyTorch. Se corre en dos etapas. Al lado de `practica_4_1.py` tienen que estar `algoritmos.py` y la carpeta `datos/`.

```
MPLBACKEND=Agg python practica_4_1.py resultados
MPLBACKEND=Agg python practica_4_1.py figuras
```

La primera etapa corre todos los algoritmos y tarda (del orden de una o dos horas); la segunda dibuja en segundos a partir de los CSV. `MPLBACKEND=Agg` evita un error de matplotlib cuando EconML y `stochtree` dibujan desde otros hilos.

Las funciones `asignacion`, `simular`, `metricas` y `cargar_acic` se corrigen automáticamente: no cambies sus nombres ni sus argumentos. En `simular` el corrector solo mira la distribución de los datos, así que tus figuras pueden diferir un poco de las de la consigna si sorteás en otro orden (el docstring dice qué orden las reproduce exactamente). No agregues código fuera de las funciones y del bloque `if __name__ == "__main__":`, porque el corrector importa tu archivo y ese código se ejecutaría.